

Interaksjonsdesign

Prosess for interaksjonsdesign

1. **Definer krav** – avklar brukerbehov, mål og rammer for løsningen.
2. **Design alternativer** – lag flere mulige forslag til hvordan løsningen kan fungere.
3. **Lag prototype** – gjør designet konkret i en enkel eller avansert modell.
4. **Evaluer** – vurder om løsningen fungerer for brukerne og oppfyller kravene.

De fem dimensjonene i interaksjonsdesign

1. **Ord** – tekst og språk som brukes i grensesnittet.
2. **Visuelle representasjoner** – ikoner, farger, bilder, typografi og andre grafiske elementer.
3. **Fysiske objekter og rom** – den fysiske konteksten eller enheten brukeren samhandler gjennom.
4. **Tid** – hvordan interaksjonen utvikler seg over tid, for eksempel animasjoner, lyd og responstid.
5. **Atferd** – hvordan systemet oppfører seg, og hvilke reaksjoner og opplevelser dette skaper hos brukeren.

Don Normans prinsipper

1. **Synlighet** – funksjoner og muligheter skal være lette å oppdage.
2. **Feedback** – systemet skal gi tydelig tilbakemelding på hva som skjer.
3. **Begrensninger** – designet bør begrense feil handlinger og lede brukeren riktig.
4. **Avbildning (mapping)** – sammenhengen mellom kontroll og effekt bør være intuitiv.
5. **Konsistens** – like ting bør fungere likt i hele systemet.
6. **Tilbydelser / affordances** – designet skal signalisere hvilke handlinger som er mulige.

Prototyping

Dimensjoner ved prototyping

1. **Utseende** – hvordan prototypen ser ut visuelt.
2. **Innhold** – hvilken informasjon og hvilke elementer som er med.
3. **Funksjonalitet** – hvilke funksjoner som er simulert eller implementert.
4. **Interaktivitet** – hvor mye brukeren faktisk kan gjøre i prototypen.

5. **Romlig struktur** – hvordan skjermbilder, elementer eller deler av løsningen er organisert.

Klasser av prototyping

- **Horisontal prototype** – bred prototype med mange funksjonsområder, men lite dybde.
- **Vertikal prototype** – smal prototype som går i detalj og dybde på én funksjon.

Aspekter ved utvikling av prototyper

- **Evolusjonær prototype** – prototype som videreutvikles til en del av sluttproduktet.
- **Bruk-og-kast-prototype** – prototype som brukes til læring og deretter forkastes.
- **Lavoppløselig prototype** – enkel prototype med lite detalj og lite interaksjon, for eksempel papirskisse.
- **Høyoppløselig prototype** – mer realistisk prototype med mye detalj og ofte mer interaksjon.

Evaluerings

Forskjell: testing vs evaluering

- **Testing** – verifisering av om systemet fungerer teknisk riktig.
- **Evaluerings** – vurdering av om løsningen er nyttig, brukbar og riktig for behovet.

Evalueringsformer

- **Formativ evaluering** – evaluering underveis for å forbedre løsningen mens den utvikles.
- **Summativ evaluering** – evaluering av ferdig løsning for å vurdere sluttresultatet.

Evaluerings som involverer brukere

- **Brukertest** – brukere utfører oppgaver mens man observerer problemer og atferd.
- **A/B-test** – to eller flere varianter sammenlignes for å se hvilken som fungerer best.
- **Feltstudie** – evaluering i brukerens naturlige omgivelser.
- **Opportunistisk evaluering** – rask tilbakemelding fra brukere på en idé eller problemstilling.

Evaluerings som ikke involverer brukere

- **Heuristisk gjennomgang** – eksperter vurderer løsningen ut fra etablerte brukbarhetsprinsipper.

- **Prediktive modeller** – analytiske eller beregnende modeller brukes for å forutsi brukeropplevelse eller ytelse.

Evaluering som kan involvere både eksperter og brukere

- **Pluralistisk gjennomgang** – brukere, utviklere og eksperter går sammen gjennom scenarioer og vurderer løsningen.

Evalueringsmetoder

- **Observasjon** – se hvordan brukeren faktisk oppfører seg.
- **Spørre brukere** – samle innsikt gjennom intervju, samtale eller spørreskjema.
- **Direkte testing** – brukeren prøver konkrete oppgaver i løsningen.
- **Spørre eksperter** – hente vurderinger fra fagpersoner.
- **Modellering** – bruke modeller for å analysere eller forutsi bruk.

Evalueringsteknikker / hva man kan teste

- **Kodevalidering** – sjekk av om kode og struktur følger krav eller standarder.
- **Testing i ulike nettlesere og på ulike plattformer** – sjekk av kompatibilitet på tvers av miljøer.
- **Testing av fleksibiliteten til et design** – vurdering av hvor godt designet fungerer under ulike forhold.
- **Testing av grad av universell utforming / brukskvalitet** – sjekk av tilgjengelighet og brukervennlighet.
- **Testing av feilmeldinger** – vurdering av om feilmeldinger er tydelige og hjelpsomme.

Brukertesting

Kjennetegn ved god brukertesting

- **Konkret hensikt** – testen skal ha et tydelig mål.
- **Avgrenset formål** – man tester noe spesifikt, ikke “alt på en gang”.
- **Realistiske oppgaver** – oppgavene bør ligne ekte brukssituasjoner.
- **Observasjon av brukeren** – man ser hva brukeren faktisk gjør, ikke bare hva brukeren sier.

Universell utforming

Prinsipper for universell utforming

1. **Enkel og intuitiv i bruk** – løsningen skal være lett å forstå.
2. **Forståelig informasjon** – nødvendig informasjon skal kommuniseres tydelig.
3. **Toleranse for feil** – designet skal redusere risiko og konsekvenser av feil.
4. **Like muligheter for alle** – løsningen skal kunne brukes av personer med ulike ferdigheter.
5. **Fleksibel i bruk** – løsningen skal støtte ulike preferanser og måter å bruke systemet på.
6. **Lav fysisk anstrengelse** – løsningen skal kunne brukes med minst mulig belastning.
7. **Størrelse og plass for tilgang og bruk** – fysisk og visuell utforming skal støtte tilgang og bruk.

Hva “størrelse og plass for tilgang og bruk” skal muliggjøre

1. **Tilgang** – brukeren skal kunne nå og oppfatte løsningens elementer.
2. **Rekkevidde** – brukeren skal kunne komme til funksjonene uavhengig av kroppslige forutsetninger.
3. **Betjening og bruk** – løsningens elementer skal kunne brukes praktisk og sikkert.

Gode argumenter for universell utforming

- **Mer inkluderende** – flere mennesker kan bruke løsningen.
- **Enklere å bruke** – god struktur og navigasjon hjelper alle brukere.
- **Mer mobiltilpasset** – tilgjengelige løsninger fungerer ofte bedre på små skjermer.
- **Mer synlig for Google / bedre SEO** – god struktur og semantikk hjelper søkemotorer.
- **Mer lønnsomt** – flere brukere når fram, og supportbehov kan reduseres.

WCAG

De fire prinsippene i WCAG

1. **Mulig å oppfatte (Perceivable)** – innhold må kunne oppfattes av brukeren.
2. **Mulig å betjene (Operable)** – grensesnittet må kunne brukes og navigeres.
3. **Forståelig (Understandable)** – innhold og funksjonalitet må være forståelige.
4. **Robust (Robust)** – løsningen må fungere med ulike nettlesere og hjelpemidler.

Eksempler på krav i WCAG

- **Innhold skal være tilgjengelig via tastatur** – brukeren skal ikke være avhengig av mus.
- **Bilder skal ha alternativ tekst** – skjermlesere må kunne formidle bildeinnhold.
- **Video skal ha teksting eller alternativ lyd** – innholdet må være tilgjengelig ved ulike behov.
- **Skjema og formater skal være lette å forstå** – brukerens oppgaver må være tydelige.
- **Innhold skal være lett å forstå** – språk og struktur må støtte forståelse.

UML

Vanlige UML-modeller / diagramtyper i pensum

- **Use case-diagram** – viser aktører og hvilke mål eller funksjoner de bruker systemet til.
- **Domenemodell** – viser sentrale begreper i domenet og relasjonene mellom dem.
- **Sekvensdiagram** – viser meldinger og samspill mellom deltakere over tid.
- **Aktivitetsdiagram** – viser flyt av aktiviteter, beslutninger og parallelle handlinger.
- **Klassediagram** – viser klasser, attributter, metoder og relasjoner.
- **Pakkediagram** – viser hvordan elementer grupperes i pakker eller moduler.
- **Tilstandsmaskindiagram** – viser tilstander og overganger for et objekt eller system.

Elementer i sekvensdiagram

- **Deltakere** – aktører, objekter eller komponenter som deltar i samspillet.
- **Meldinger / funksjonskall** – kommunikasjon mellom deltakerne.
- **Levetidslinje** – stiptet linje som viser deltakerens eksistens over tid.
- **Aktiveringsboks** – viser når en deltaker utfører arbeid.
- **Rammer (f.eks. løkke/alt)** – markerer kontrollstrukturer som valg eller repetisjon.

Elementer i aktivitetsdiagram

- **Aksjoner** – konkrete handlinger eller steg i en prosess.
- **Initial node** – startpunktet for aktiviteten.
- **Final node** – sluttpunktet for aktiviteten.
- **Decision node** – punkt der prosessen forgrenes ut fra betingelser.
- **Fork node** – deler prosessen i parallelle løp.
- **Join node** – samler parallelle løp igjen.
- **Merge node** – slår sammen alternative flyter.

- **Overganger / piler** – viser rekkefølge og flyt mellom aktiviteter.

Elementer i klassediagram

- **Klasse** – en type objekt med data og atferd.
- **Attributter** – data som hører til klassen.
- **Metoder** – funksjoner eller operasjoner klassen kan utføre.

Relasjoner i klassediagram

- **Avhengighet (dependency)** – en svak kobling der én klasse bruker en annen.
- **Assosiasjon** – en strukturell kobling mellom to klasser.
- **Aggregering** – hel-del-relasjon der delen kan eksistere uten helheten.
- **Komposisjon** – sterk hel-del-relasjon der delen hører tett til helheten.
- **Arv / generalisering** – en subklasse arver fra en superklasse.
- **Realisering** – en klasse implementerer et grensesnitt.

Hovedelementer i tilstandsmaskindiagram

- **Starttilstand** – tilstanden systemet eller objektet starter i.
- **Sluttilstand** – tilstanden der livsløpet avsluttes.
- **Tilstander** – ulike situasjoner objektet kan befinne seg i.
- **Overganger** – bevegelser fra én tilstand til en annen.
- **Triggere / hendelser** – det som utløser en overgang.

Brukerhistorier

Standardformat for brukerhistorier

- **Som [rolle]** – beskriver hvem brukeren er.
- **vil jeg [mål/ønske]** – beskriver hva brukeren ønsker å gjøre.
- **slik at [nytte/verdi]** – beskriver hvorfor dette er viktig.

Programvarearkitektur

Hvorfor programvarearkitektur er viktig

- **Forstå systemet tidlig** – gjør det lettere å se helheten før implementasjon.
- **Redusere kostnader** – tidlige gode valg sparer tid og penger senere.
- **Unngå duplisering av ressurser** – tydelig struktur reduserer overlappende løsninger.
- **Øke kvalitet** – bedre struktur gir bedre vedlikeholdbarhet og testbarhet.
- **Hjelp risikostyring** – man oppdager tekniske utfordringer tidligere.
- **Støtte kommunikasjon mellom interessenter** – gir et felles bilde av systemet.
- **Gi retningslinjer for utviklere** – hjelper teamet å utvikle konsistent.

Kompleksitetshåndtering

- **Splitt og hersk** – del opp problemet i mindre deler.
- **Lav kobling** – delene bør være minst mulig avhengige av hverandre.
- **Mer testbar kode** – mindre og tydeligere deler er enklere å teste.
- **Mer robust kode** – isolerte deler tåler endringer bedre.
- **Enklere delegering av ansvar** – team kan jobbe med ulike deler parallelt.

Trelagsarkitektur

De tre lagene

1. **Presentasjonslag** – viser informasjon og håndterer brukerinteraksjon.
2. **Forretningslogikklag** – utfører regler, prosessering og kjernefunksjonalitet.
3. **Data access-lag** – håndterer lagring, henting og oppdatering av data.

Hovedidé i lagdeling

- **Hvert lag har spesifikt ansvar** – gjør systemet mer oversiktlig.
- **Hvert lag yter tjenester til laget over / bruker laget under** – gir strukturert avhengighet.
- **Gir lav kobling** – endringer i ett lag påvirker ikke nødvendigvis alle andre.

Master-slave pattern

Hoveddeler

- **Master** – styrer, fordeler arbeid og samler resultater.
- **Slaver** – utfører arbeid delegert av master.

Fordeler

- **Tydelig oversikt i master** – gir sentral kontroll over prosessen.
- **Slavene kan enhetstestes separat** – hver del kan testes isolert.
- **Master kan integrasjonstestes mot slavene** – samspillet kan testes samlet.

Event-bus pattern

Fire hovedkomponenter

1. **Event listener** – lytter etter hendelser.
2. **Event source** – produserer eller sender hendelser.
3. **Channel** – kanal eller emne der hendelser publiseres.
4. **Event bus** – infrastrukturen som frakter hendelser mellom kilder og lyttere.

MVC

Tre deler i MVC

1. **Model** – data og forretningslogikk.
2. **View** – det brukeren ser i grensesnittet.
3. **Controller** – håndterer input og styrer flyten mellom model og view.

Fordeler

- **Høyere kohesjon** – hver del får tydeligere ansvar.
- **Gjenbrukbarhet** – deler kan lettere brukes på nytt.

- **Skiller ansvar** – gjør systemet enklere å forstå og endre.

Mikrotjenester

Mikrotjenester er basert på

- **Single Responsibility Principle** – hver tjeneste bør ha ett tydelig hovedansvar.

Kjennetegn

- **Selvstendige tjenester** – hver tjeneste kan utvikles og driftes separat.
- **Én hovedoppgave per tjeneste** – tydelig ansvarsdeling.
- **Egen data** – tjenesten eier vanligvis sin egen datalagring.
- **Eget kjøremiljø** – tjenesten kan kjøres og deployes uavhengig.

Fordeler

- **Skalerbart** – tjenester kan skaleres separat.
- **Modulært** – systemet deles i mindre moduler.
- **Fleksibel teknologibruk** – ulike tjenester kan bruke ulik teknologi.
- **Høy tilgjengelighet / feil isoleres** – feil i én tjeneste stopper ikke nødvendigvis hele systemet.
- **Støtter kontinuerlig levering (CI/CD)** – tjenester kan oppdateres uavhengig.
- **Passer DevOps** – tett kobling mellom utvikling, drift og deploy.

Ulemper

- **Kompleks debugging** – feilsparing på tvers av tjenester er vanskeligere.
- **Kompleks logging på tvers av tjenester** – data om feil og flyt spres i flere systemer.
- **Krever automatisering** – bygg, test og deploy må ofte automatiseres sterkt.
- **Vanskelig å teste distribuerte systemer** – mange avhengigheter og miljøer kompliserer test.
- **Datainkonsistens kan oppstå** – data er fordelt og kan bli ute av synk.
- **Økt ressursforbruk** – mange tjenester krever mer infrastruktur.

Monolittarkitektur

Fordeler

- **God gjennomstrømning / lite overhead** – mindre kommunikasjon mellom separate tjenester.
- **Enklere å teste og debugge i starten** – alt er samlet i én applikasjon.

Ulemper

- **Vanskeligere å vedlikeholde ved vekst** – systemet blir tungt å endre når det blir stort.
- **Lite skalerbar** – hele systemet må ofte skales samlet.
- **Lite utvidbar sammenlignet med modulære løsninger** – struktur og vekst blir vanskeligere.
- **Tett koblet** – delene er ofte avhengige av hverandre.
- **Feil i én del kan stoppe hele systemet** – svak isolasjon av feil.

Plugin-basert arkitektur

Hoveddeler

- **Kjerne-system** – grunnsystemet som gir basisfunksjonalitet.
- **Plugin-moduler / extensions** – tillegg som utvider funksjonaliteten.

Fordeler

- **Lav kobling** – plugins kan være relativt uavhengige av hverandre.
- **Høy isolasjon mellom funksjoner** – endringer i én plugin trenger ikke påvirke andre.
- **Fleksibelt** – funksjonalitet kan legges til eller fjernes.
- **Utvidbart** – systemet kan vokse gjennom nye plugins.

Ulemper

- **Endringer i kjernen kan ødelegge plugins** – bakoverkompatibilitet blir viktig.
- **Mange plugins kan redusere ytelsen** – ekstra moduler kan gi mer belastning.
- **Kompleks testing av hele systemet** – mange kombinasjoner må testes.

Eksempler

- **Chrome Web Store** – nettleserutvidelser.
- **IntelliJ** – IDE med mange utvidelser.
- **VS Code** – editor med plugin-system.
- **Eclipse** – utviklingsmiljø bygget rundt plugins.

Deployment-arkitekturer

Tradisjonell deployment

- **Applikasjoner kjøres direkte på fysiske servere** – programvaren installeres rett på maskinvaren.

Utfordringer ved tradisjonell deployment

- **Ressursallokering kan bli ineffektiv** – det er vanskelig å fordele kapasitet godt.
- **Én applikasjon kan ta for mye kapasitet** – andre systemer kan påvirkes negativt.

Virtualisering (VMs)

- **Flere virtuelle maskiner på én fysisk server** – én fysisk maskin deles opp logisk.
- **Hver VM har eget operativsystem** – gir sterk isolasjon mellom miljøene.

Ulempe ved virtualisering

- **Ressurser reserveres selv om de ikke brukes** – effektiviteten kan bli lav.

Container-deployment

- **Containere deler OS, men er isolerte** – lettere enn virtuelle maskiner, men fortsatt avgrenset.
- **Lettere enn virtuelle maskiner** – mindre overhead og raskere oppstart.

Nøkkelteknologier i container-deployment

- **Docker** – teknologi for å bygge og kjøre containere.
- **Kubernetes** – plattform for orkestrering og styring av containere.

Fordeler med containere

- **Enkel distribusjon** – lett å flytte og kjøre samme løsning flere steder.
- **Konsistent kjøremiljø** – “det fungerer likt overalt”.
- **Løst koblet** – tjenester kan holdes adskilt.
- **Distribuert** – egnet for moderne distribuerte systemer.
- **Elastisk** – kapasitet kan justeres dynamisk.

Systemutviklingsmetodikk

Før smidige metoder

- **Vannfallsmodellen** – sekvensiell modell med faste faser.
- **Unified Process (UP)** – iterativ prosess med mye modellering og struktur.

Grunnleggende prinsipper i smidig metode

1. **Problem based** – store problemer brytes ned i mindre deler.
2. **Collaborative** – flere roller samarbeider tett gjennom hele prosessen.
3. **Iterativ** – løsningen utvikles og forbedres i gjentatte runder.
4. **Adaptiv** – planen og løsningen tilpasses ny læring og endring.

Agile Manifesto – fire hovedverdier

1. **Individer og interaksjoner fremfor prosesser og verktøy** – mennesker og samarbeid er viktigst.
2. **Fungerende programvare fremfor omfattende dokumentasjon** – praktisk verdi prioriteres.
3. **Kundesamarbeid fremfor kontraktsforhandlinger** – kontinuerlig dialog er viktigere enn låste avtaler.
4. **Reagere på endring fremfor å følge en plan** – tilpasning er viktigere enn rigid planfølging.

Extreme Programming (XP)

Kjennetegn

- **Lite dokumentasjon** – fokus på kode og praksis fremfor tunge dokumenter.
- **Lite modellering** – mindre vekt på omfattende forhåndsmodellering.
- **Tett feedback-loop** – raske tilbakemeldinger fra tester, kode og kunde.
- **Kort iterasjonstid** – små og hyppige leveranser.

Antall faste regler i XP

- **29 regler** – XP beskrives ofte som en metode med mange konkrete arbeidsregler.

Eksempler på XP-regler

- **Parprogrammering** – to utviklere jobber sammen ved samme oppgave.
- **Skriv enhetstester før kode** – test først for å styre utviklingen.
- **Integrer én liten endring av gangen** – små endringer gjør feil lettere å oppdage.
- **Enkel og tydelig kode** – løsningen skal være så enkel som mulig.
- **Hyppig refaktorering** – forbedre kode kontinuerlig uten å endre atferd.
- **Korte feedback-looper** – rask respons på endringer og feil.
- **Hyppige tester** – kvalitet kontrolleres kontinuerlig.
- **Hyppige leveranser** – verdi leveres ofte.

Fem sentrale verdier i XP

1. **Communication** – tydelig kommunikasjon i teamet.
2. **Simplicity** – hold løsningen enkel.
3. **Feedback** – lær raskt av tester og brukere.
4. **Courage** – tør å endre og forbedre kode når det trengs.
5. **Respect** – respekt for teamet, koden og arbeidet.

Test-Driven Development (TDD)

Grunnidé

1. **Skriv test først** – definer forventet oppførsel.
2. **Kjør testen og se at den feiler** – bekreft at testen faktisk fanger mangel på funksjonalitet.
3. **Skriv kode som får testen til å bestå** – implementer minimum som trengs.
4. **Refaktorer** – forbedre koden uten å endre funksjonen.

Fokus i TDD

- **Primært enhetstesting** – små isolerte funksjoner eller klasser.
- **Noen ganger integrasjonstesting** – i enkelte tilfeller kan TDD brukes bredere.
- **Sjelden systemtesting** – TDD brukes oftest nær kodenivå.

Fordeler

- **Bedre kodekvalitet** – design blir ofte ryddigere.
- **Høyere testdekning** – flere deler av koden får tester.
- **Rask tilbakemelding** – feil oppdages tidlig.
- **Økt tillit** – lettere å endre kode når testene beskytter funksjonaliteten.
- **Testene fungerer som dokumentasjon** – viser hva systemet skal gjøre.

Ulemper

- **Lærekurve** – krever ny måte å tenke på.
- **Tidsbruk til å skrive tester** – særlig i starten.
- **Vedlikehold av tester** – testene må oppdateres når systemet endres.

Behavior-Driven Development (BDD)

Hovedidé

- **Kobler forretningsmål og teknologi** – teknisk utvikling knyttes til bruker- og forretningsverdi.
- **Beskriver systematferd fra brukerens perspektiv** – fokus på hva systemet skal gjøre for brukeren.

- **Bruker naturlig språk** – krav uttrykkes i språk flere kan forstå.

Given-When-Then-formatet

1. **Given (Gitt)** – beskriver starttilstand eller kontekst.
2. **When (Når)** – beskriver handlingen eller hendelsen.
3. **Then (Så)** – beskriver forventet resultat.

Nøkkelelementer i Gherkin

- **Feature** – overordnet funksjonalitet eller egenskap.
- **Scenario** – konkret eksempel på oppførsel.
- **Given** – forutsetning.
- **When** – handling.
- **Then** – forventning.
- **And** – ekstra steg som bygger videre på de andre.

Hva BDD bidrar til

- **Tydeligere akseptkriterier** – lettere å forstå hva som må være oppfylt.
- **Felles forståelse mellom forretning og utvikling** – mindre misforståelser.
- **Testbar spesifikasjon** – krav kan omsettes til tester.
- **Høyere fokus på brukerens perspektiv** – atferd sees fra sluttbrukerens side.

Scrum

Hva Scrum er

- **Tidsboksbasert smidig rammeverk** – arbeid organiseres i faste perioder kalt sprinter.

Fokus i Scrum

- **Kryssfunksjonelle team** – teamet skal samlet ha kompetansen som trengs.
- **Sprinter** – korte, faste iterasjoner.
- **Inkrementell fremgang** – produktet forbedres litt etter litt.

Scrum-seremonier

1. **Sprint planning** – planlegging av hva som skal gjøres i sprinten.
2. **Daily standup / daily scrum** – kort daglig møte for koordinering.
3. **Sprint review** – gjennomgang av resultatet med interessenter.
4. **Retrospektiv** – refleksjon over hvordan teamet kan forbedre arbeidsprosessen.

Typiske artefakter / objekter i Scrum

- **Product backlog** – prioritert liste over alt som kan gjøres i produktet.
- **Sprint backlog** – utvalget av arbeid som tas inn i en sprint.
- **Product backlog items (PBI)** – enkeltenheter i product backlog.
- **Scrum board** – visuell tavle for oppgaver i sprinten.

Lean

Lean-prinsipper

1. **Eliminate waste** – fjern arbeid som ikke gir verdi.
2. **Build quality in** – kvalitet skal bygges inn i prosessen, ikke bare sjekkes til slutt.
3. **Create knowledge** – læring og tilbakemelding skal brukes aktivt.
4. **Defer commitment** – vent med irreversible valg til man vet mer.
5. **Deliver fast** – lever verdi raskt.
6. **Respect people** – stol på og støtt menneskene i systemet.
7. **Optimize the whole** – se helheten, ikke bare enkeltledd.

Kanban

Kjennetegn

- **Flytbasert** – fokus på jevn gjennomstrømning av arbeid.
- **Kontinuerlig levering** – arbeid leveres fortløpende.
- **Ingen faste iterasjoner** – i motsetning til Scrum er det ikke sprinter som styrer.
- **Visuell styring** – tavler brukes for å se status og flyt.

Viktige elementer

- **Kanban-board** – tavle som visualiserer arbeidsflyten.
- **Begrensning av arbeid i prosess (WIP)** – setter grenser for hvor mange oppgaver som kan være i arbeid samtidig.
- **Fokus på flaskehalser** – prosessen forbedres der arbeidet stopper opp.

Formål med WIP-begrensning

- **Synliggjøre flaskehalser** – gjør problemer i flyten tydelige.
- **Forbedre flyt** – reduserer opphopning av arbeid.
- **Hjelpe teammedlemmer å bidra der det stopper opp** – fremmer samarbeid på tvers.

Domain-Driven Design (DDD)

Kjennetegn

- **Høyt fokus på domenet** – utviklingen tar utgangspunkt i problemområdet.
- **Koden skal gjenspeile virkeligheten** – modell og kode bør bruke samme begreper som domenet.
- **Egnet for skalering og endring** – god domeneforståelse gjør videreutvikling lettere.
- **Fokus på det viktigste i domenet** – prioriterer kjernebegreper og kjerneprosesser.

DevOps

Hva DevOps kombinerer

- **Utvikling (Dev)** – bygging og forbedring av programvare.
- **Drift (Ops)** – utrulling, overvåking og vedlikehold i drift.

DevOps skal sikre

- **Kontinuerlig integrasjon** – kode integreres ofte.
- **Kontinuerlig levering / deploy** – kode kan leveres raskt og trygt.
- **Tettere samarbeid mellom utvikling og drift** – færre siloer mellom team.

CI (Continuous Integration)

- **Hyppig integrasjon til felles gren** – små endringer flettes ofte inn.
- **Automatisk bygging** – systemet bygges automatisk ved endringer.
- **Automatisk testing** – tester kjøres automatisk som del av bygget.

CD (Continuous Delivery / Deployment)

- **Automatisk levering / deploy av kodeendringer** – endringer kan settes i drift med mindre manuelt arbeid.

DevOps-verktøy i pensum

- **GitLab / GitHub** – versjonskontroll og samarbeid.
- **Jira / Confluence** – planlegging og dokumentasjon.
- **Maven** – byggverktøy for Java.
- **Gradle** – fleksibelt byggverktøy.
- **Jenkins** – automatiserings- og CI-server.
- **JUnit** – testverktøy for Java.
- **Selenium** – verktøy for automatisert system-/UI-testing.
- **Kubernetes** – orkestrering av containere.
- **Helm** – pakkebehandling for Kubernetes.
- **Docker** – containerteknologi.
- **Kibana** – visualisering av logger og søk.
- **Grafana** – visualisering av målinger og monitorering.

REST

REST-prinsipper i pensum

- **Klient-tjener** – klient og server har ulike roller og ansvar.
- **Ressurser identifiseres via URI-er** – data og funksjonalitet modelleres som ressurser.
- **Ressurser manipuleres via HTTP-metoder** – standardiserte operasjoner brukes for å hente og endre data.

HTTP-metoder i pensum

- **GET** – hente data.
- **POST** – opprette ny ressurs.
- **PUT** – oppdatere eller erstatte ressurs.
- **DELETE** – slette ressurs.

Testing

Hvorfor teste

- **Sikre riktig respons ved input/interaksjon** – systemet må reagere riktig på brukerhandlinger.
- **Finne bugs** – feil må oppdages før brukerne gjør det.
- **Verifisere ytelse under edge cases** – systemet må tåle vanskelige situasjoner.
- **Vurdere brukervennlighet** – løsningen må være god å bruke.
- **Kontrollere at krav er oppfylt** – systemet må gjøre det som er avtalt.
- **Kontrollere at systemet kan installeres i riktig miljø** – løsningen må fungere der den skal kjøres.
- **Kontrollere at systemet tåler endringer** – videreutvikling må ikke ødelegge eksisterende funksjonalitet.

Testing principles

1. **Testing shows the presence of defects** – testing kan vise at feil finnes, ikke bevise at ingen feil finnes.
2. **Exhaustive testing is impossible** – det er umulig å teste absolutt alt.
3. **Early testing** – testing bør starte tidlig.
4. **Defects clustering** – mange feil samler seg ofte i få deler av systemet.
5. **Pesticide paradox** – samme tester finner færre nye feil over tid.
6. **Testing is context dependent** – teststrategi må tilpasses situasjonen.
7. **Absence of error fallacy** – et feilfritt system er ikke nyttig hvis det løser feil problem.

Tradisjonell testing

- **Utvikling og testing skjer separat** – et team lager systemet, et annet tester det.
- **QA-team kvalitetssikrer etter utvikling** – test kommer ofte sent i prosessen.

Ulempe ved tradisjonell testing

- **Mye tid går til kommunikasjon og overlevering** – feil blir dyrere og tregere å håndtere.

Smidig testing

- **Utviklere tester egen kode kontinuerlig** – test skjer parallelt med utvikling.
- **Små inkremitter** – mindre endringer testes og leveres ofte.
- **Automatiserte tester brukes mye** – rask feedback er viktig.

Fordeler ved smidig testing

- **Raskere feedback** – feil oppdages tidlig.
- **Tettere kobling mellom utvikling og kvalitet** – kvalitet bygges inn underveis.
- **QA kan fokusere på viktige scenarioer** – manuell test kan brukes der den gir mest verdi.

Automatiserte tester

Når de ofte kjøres

- **Ved opprettelse av pull request** – nye endringer testes før sammenslåing.
- **I CI-pipelines** – test inngår i automatisert byggflyt.
- **Ved nye kodeendringer** – hyppig kjøring gir rask tilbakemelding.

Kjennetegn

- **Enhetstester og integrasjonstester kjøres automatisk** – vanlige testnivåer for automatisering.
- **Resultater rapporteres raskt** – utviklere får rask oversikt over feil.

Fordeler

- **Rask feedback** – feil oppdages tidlig.
- **Høy hastighet** – mange tester kan kjøres fort.
- **Kontinuerlig kontroll** – kvalitet følges opp automatisk.
- **Forutsigbar kvalitet** – samme test kjøres likt hver gang.
- **Kostnadsbesparende over tid** – reduserer manuelt repeterende arbeid.

Ulemper / utfordringer

- **Høy oppstarts- og vedlikeholdskostnad** – testene må bygges og holdes oppdatert.
- **Krever riktig fokus og god testdesign** – dårlige automatiserte tester gir liten verdi.

Egne test-/staging-miljøer

Kjennetegn

- **Testmiljø speiler produksjonsmiljø** – test skjer i omgivelser som ligner virkeligheten.
- **Hele applikasjonen kan testes før release** – gir helhetlig kontroll.

Fordeler

- **Mer realistisk testing** – større sjanse for å finne reelle feil.
- **QA kan teste hele applikasjonen** – også på systemnivå.
- **Infrastructure as Code gjør oppsett enklere** – miljøer kan opprettes og vedlikeholdes systematisk.

CI og små endringer

Fordeler

- **Mindre diff mellom gammel og ny versjon** – endringer blir lettere å forstå.
- **Enklere isolering av feil** – lettere å finne hva som gikk galt.
- **Enklere sporing av endringer** – bedre oversikt over historikk.
- **Lavere sannsynlighet for feil** – små endringer er lettere å kontrollere.
- **Enklere rollback** – enklere å rulle tilbake en liten endring.

Prioritering av testing

Viktige prinsipper

- **Finn riktig nivå på testdekning basert på risiko** – ikke alt må testes like mye.
- **Prioriter tester som gir størst verdi** – test der konsekvensen av feil er størst.
- **Prioriter tester som reduserer mest usikkerhet** – test der du vet minst eller er mest utsatt.

Risk poker

Hva man vurderer

- **Risiko** – sannsynlighet for at noe går galt.
- **Konsekvens** – hvor alvorlig det blir hvis det går galt.
- **Risikofaktorer** – forhold som påvirker sannsynlighet eller konsekvens.

Fordeler

- **Hele teamet deltar** – flere perspektiver tas med.
- **Bedre treffsikkerhet** – samlet vurdering blir ofte bedre enn individuell.
- **Felles forståelse av risiko** – teamet blir enige om hva som er kritisk.

Testdata

Egenskaper ved gode testdata

- **Samme data hver gang** – testresultater kan sammenlignes.
- **Deterministisk prosess** – samme input gir samme output.
- **Forutsigbart testmiljø** – eksterne tilfeldigheter påvirker minst mulig.

Fordeler

- **Enklere å isolere feil** – avvik blir lettere å forklare.
- **Mer presise testresultater** – resultatene blir mer pålitelige.

- **Bedre analyse av testutfall** – utviklere kan tolke funn mer sikkert.

Mocking

Hva mocking brukes til

- **Erstatte ekte og komplekse avhengigheter** – bytte ut systemer som er vanskelige eller upraktiske å bruke i test.
- **Isolere klasser / moduler i test** – teste én del uten ekte avhengigheter.

Typiske avhengigheter som kan mocket

- **Database** – unngå ekte database i test.
- **Eksternt API** – unngå nettverkskall og ustabile svar.
- **Betalingstjeneste** – simulere betaling uten ekte transaksjoner.

Hva som gjør mocking ideelt

- **Deterministiske svar** – testene får samme respons hver gang.
- **Samme grensesnitt som ekte avhengighet** – kan byttes inn uten å endre resten av koden.
- **Kombineres godt med dependency injection** – avhengigheter kan settes inn ved kjøring.

Fordeler

- **Muliggjør isolert testing** – én del kan testes alene.
- **Gjør automatiserte tester mulige i flere situasjoner** – flere deler blir testbare.
- **Gir kontroll over testmiljøet** – reduserer usikkerhet i testkjøringen.

Testfaser

Faser i en test

1. **Oppsett** – forbered testdata og testmiljø.
2. **Gjennomføring** – utfør handlingene testen beskriver.
3. **Verifisering** – kontroller at resultatet ble som forventet.

4. **Opprydding** – fjern eller nullstill ressurser for neste test.

Enhetstesting

Kjennetegn

- **Tester individuelle klasser / funksjoner** – små deler testes isolert.
- **Fokus på små enheter** – enkel funksjonalitet står i sentrum.
- **Ofte utviklers ansvar** – utvikleren skriver og vedlikeholder testene.

Fordeler

- **Automatisert** – enkelt å kjøre ofte.
- **Repeterbar** – gir like resultater ved samme input.
- **Kjøres i byggeprosessen** – naturlig del av utviklingsflyten.

Mål

- **Finne feil tidlig** – feil fanges før de sprer seg.
- **Stoppe feil før integrasjon med andre deler** – hindrer at små feil blir større problemer.

Integrasjonstesting

Kjennetegn

- **Tester samspill mellom komponenter** – ser på hvordan delene virker sammen.
- **Bygger på at delene allerede er testet individuelt** – enhetstesting bør være gjort først.

Mål

- **Oppdage feil i grensesnitt og samhandling** – avdekke problemer i koblinger mellom delene.

Valg

- **Kan automatiseres** – mange integrasjonstester egner seg for automatisering.

- **Bør automatiseres der det er hensiktsmessig** – særlig ved hyppige og kritiske samspill.

Systemtesting

Kjennetegn

- **Tester hele systemet** – systemet ses som en helhet.
- **Sjekker at alle deler fungerer sammen** – helhetlig kvalitet kontrolleres.

Fordeler

- **Finner feil mellom deler av systemet** – noen problemer oppstår bare i helheten.
- **Gir trygghet før levering** – bedre grunnlag for release.

Mål

- **Sjekke at systemet oppfyller krav** – både funksjonelle og ofte ikke-funksjonelle krav.
- **Finne feil før bruk** – redusere risiko før sluttbruker møter løsningen.

Valg

- **Kan utføres med verktøy som Selenium** – mulig å automatisere deler av helheten.
- **Kan være vanskelig å gjøre helt repeterbar** – mange avhengigheter påvirker resultatet.

Akseptansetesting

Kjennetegn

- **Type systemtesting** – skjer på høyt nivå i systemet.
- **Utføres for å verifisere at krav er oppfylt** – spesielt mot kundens eller brukerens forventninger.

Viktig i BDD

- **Bygger på user stories** – kobling mellom krav og test.
- **Kan utvikles sammen med kunden** – gir bedre samsvar med behov.

Kan være

- **Manuell** – mennesker går gjennom scenarier.
- **Automatisert** – scenarioer kan kjøres som automatiske tester.

Regresjonstesting

Hva det er

- **Tester at endringer ikke ødelegger eksisterende funksjonalitet** – gammel funksjon må fortsatt virke etter endringer.

Når det skjer

- **Før og etter endringer** – for å sammenligne oppførsel.
- **Kontinuerlig i CI ved innsjekk av kode** – skjer ofte automatisk i moderne utvikling.

Hvilke nivåer det kan finnes på

- **Enhetstestnivå** – små deler testes på nytt.
- **Integrasjonsnivå** – samspill testes på nytt.
- **Akseptansenivå** – brukerorientert oppførsel testes på nytt.

Valg

- **Egner seg godt for automatisering** – samme tester kjøres mange ganger.

Smoke-testing og sanity-testing

Smoke-testing

- **Tester kritisk funksjonalitet** – sjekker at systemets viktigste deler virker.
- **Rask sjekk av om systemet “i det hele tatt fungerer”** – avgjør om videre testing er verdt å starte.

Sanity-testing

- **Tester ny eller endret funksjonalitet** – fokuserer på siste endring.
- **Rask sjekk av om en spesifikk endring virker fornuftig** – smalere enn smoke-testing.

Hvorfor de er nyttige

- **Avdekker tidlig om videre testing er hensiktsmessig** – sparer tid hvis systemet er for ustabilt.

Destruktiv testing og utforskende testing

Destruktiv testing

- **Kreativ testing for å få systemet til å feile** – tester bevisst grensene.
- **Kan ikke automatiseres** – bygger på menneskelig kreativitet.
- **Krever menneskelig erfaring og kreativitet** – testeren må tenke utradisjonelt.

Utforskende testing

- **Tester med aktiv læring og utforskning underveis** – testdesign og gjennomføring skjer samtidig.
- **Bygger på innsikt i systemet og nye hypoteser** – man følger spor som dukker opp.

Forskjell

- **Destruktiv testing: fokus på å bryte systemet** – målet er å finne svakheter ved å presse løsningen.
- **Utforskende testing: bredere utforskning og læring** – målet er både forståelse og feilfunn.

Usabilitytesting

Hva det tester

- **Brukervennlighet** – hvor lett systemet er å bruke.
- **GUI** – hvordan det grafiske brukergrensesnittet fungerer.
- **Om brukeren / kunden er fornøyd** – opplevd kvalitet og tilfredshet.

Ytelsestesting

Hva ytelsestesting undersøker

- **Hvor mye last systemet tåler** – kapasitet under normal eller høy belastning.
- **Hvor mye plutselig last systemet tåler** – reaksjon på raske topper.
- **Ytelse over tid** – stabilitet og ressursbruk ved langvarig drift.
- **Lekkasjer / ressursproblemer** – oppdage for eksempel minnelekkasjer.

Hvordan ytelse måles

- **Antall samtidige brukere** – hvor mange som bruker systemet samtidig.
- **Transaksjoner per tidsenhet** – hvor mye arbeid systemet utfører per tidsrom.
- **Throughput** – mengde data eller arbeid som behandles over tid.
- **Responstid** – hvor lang tid systemet bruker på å svare.

Hva flaskehalser er

- **Systemets svakeste / tregeeste punkt** – delen som begrenser total ytelse.
- **Det som begrenser total ytelse** – det som holder resten tilbake.

Vanlige årsaker til flaskehalser

- **CPU** – prosessoren er overbelastet.
- **Minne** – for lite eller ineffektiv bruk av RAM.
- **Tråder** – dårlig håndtering av parallellitet eller venting.
- **Databasetilkobling** – database eller forbindelser blir begrensningen.

Typer ytelsestester

1. **Lasttest** – måler oppførsel ved forventet eller gradvis økende last.
2. **Stresstest** – finner yttergrensen der systemet bryter sammen.
3. **Spike-test** – tester reaksjon på plutselige lasttopper.
4. **Utholdenhetstest** – tester systemet under jevn belastning over tid.

Black-box og white-box testing

Black-box testing

- **Ingen innsikt i intern kode** – testeren ser bare input og output.
- **Bygger på krav, spesifikasjoner og forventet oppførsel** – ikke på intern implementasjon.

White-box testing

- **Full innsikt i intern kode og struktur** – testeren kjenner logikken og oppbygningen.
- **Tester intern logikk og oppbygning** – dekker veier, betingelser og intern struktur.

Automatisk vs manuell testing

Fordeler ved automatiske tester

- **Raske** – kan kjøres ofte og på kort tid.
- **Nøyaktige** – samme fremgangsmåte hver gang.
- **Repeterbare** – resultatene blir konsistente.
- **Gode for kjente forventninger** – passer godt der riktig resultat er tydelig definert.

Typiske automatiserte tester

- **Enhetstester** – små isolerte kodeenheter.
- **Regresjonstester** – sjekk av at gammel funksjonalitet fortsatt virker.
- **Mange integrasjonstester** – testing av samspill mellom komponenter.
- **Mange ytelsestester** – automatiske last- og stresstester.

Ulemper ved automatiske tester

- **Krever vedlikehold** – testene må oppdateres når systemet endres.
- **Fanger ikke alt** – særlig dårlig på subjektive forhold.
- **Dårligere på kreative og uventede problemer** – mennesker er bedre til utforskning.

Fordeler ved manuelle tester

- **Kreative** – testeren kan tenke fritt og prøve uventede ting.

- **Fanger ofte naturlige brukerfeil og uventede problemer** – realistisk menneskelig bruk avdekkes bedre.
- **Gode for utforskning og opplevelse** – særlig viktig for usability og ad hoc-testing.

Typiske manuelle tester

- **Utforskende testing** – læring og test skjer samtidig.
- **Destruktiv testing** – kreativ testing for å bryte systemet.
- **Mye usabilitytesting** – vurdering av opplevelse og brukeradferd.

Testtyper som kan være både manuelle og automatiske

- **Smoke-testing** – raske helsesjekker av løsning.
- **Sanity-testing** – raske sjekker av endret funksjonalitet.
- **Integrasjonstesting** – samspill mellom komponenter.
- **Systemtesting** – helhetlig testing av systemet.
- **Akseptansetesting** – verifisering mot behov og krav.
- **Ytelsestesting** – måling av kapasitet og responstid.

GDPR

Grunnleggende prinsipper i GDPR

1. **Lovlighet, rimelighet og gjennomsiktighet** – behandling må være lovlig, rettferdig og forståelig for den registrerte.
2. **Formålsbegrensning** – data skal bare brukes til tydelig definerte formål.
3. **Dataminimering** – bare nødvendige opplysninger skal samles inn.
4. **Riktighet** – opplysninger skal være korrekte og oppdaterte.
5. **Lagringsbegrensning** – data skal ikke lagres lenger enn nødvendig.
6. **Integritet og konfidensialitet** – data skal beskyttes mot uautorisert tilgang og tap.
7. **Ansvarliggjøring** – virksomheten må kunne dokumentere at reglene følges.

De registrertes rettigheter

1. **Rett til informasjon** – få vite hvilke data som behandles og hvorfor.
2. **Rett til innsyn** – få tilgang til egne opplysninger.
3. **Rett til retting** – få uriktige opplysninger korrigert.

4. **Rett til sletting** – kreve at opplysninger fjernes i visse tilfeller.
5. **Rett til begrensning** – kreve at behandling stoppes midlertidig.
6. **Rett til å protestere** – motsette seg visse typer behandling.
7. **Rett til dataportabilitet** – få egne data ut i et maskinlesbart format.

Typiske unntak / begrensninger

- **Juridisk forpliktelse** – behandling kan være nødvendig for å oppfylle lovkrav.
- **Offentlig sikkerhet** – noen rettigheter kan begrenses av samfunnshensyn.
- **Forskning** – særregler kan gjelde ved forskning og statistikk.

Profesjonsetikk

Ingeniørens ansvar omfatter blant annet

- **Sosialt ansvar** – teknologi påvirker samfunn, arbeidsliv og mennesker.
- **Sikkerhet og pålitelighet** – systemer må være trygge og til å stole på.
- **Brukeres velferd** – løsninger må ikke skade brukerne eller deres livssituasjon.
- **AI og militær bruk / alvorlige konsekvenser av teknologi** – utviklere må reflektere over alvorlige anvendelser og ansvar.

Eksempler på NITO-prinsipper i pensum

- **Vise respekt for kolleger** – rettferdig og profesjonell behandling av andre.
- **Vedkjenne seg faglig ansvar** – stå inne for eget arbeid og egne vurderinger.
- **Følge anerkjente kvalitetsnormer** – bruke faglig gode standarder og praksiser.
- **Bidra til åpenhet om teknologiske konsekvenser** – ikke skjule risiko eller negative effekter.

IEEE/ACM Code of Ethics – hovedområder

1. **Samfunn** – arbeid i offentlighetens interesse.
2. **Kunde og arbeidsgiver** – opptre ansvarlig overfor dem man arbeider for.
3. **Produkt** – sikre høy kvalitet i programvare og løsninger.
4. **Vurdering** – utvise integritet og faglig skjønn.
5. **Ledelse** – fremme etisk ledelse i utviklingsarbeid.
6. **Profesjon** – styrke profesjonens integritet og omdømme.
7. **Kolleger** – støtte og behandle kolleger rettferdig.

8. **Egen utvikling / livslang læring** – holde seg faglig oppdatert og utvikle seg videre.

Nøkkelprinsipper i profesjonsetikken

- **Ærlighet og integritet** – vær sannferdig om begrensninger og risiko.
- **Respekt for personvern** – bygg løsninger som beskytter individet.
- **Kvalitetsbevissthet** – arbeid systematisk for god kvalitet.
- **Ansvar for konsekvenser** – vurder hvordan teknologien påvirker mennesker og samfunn.
- **Kontinuerlig vurdering** – etiske vurderinger må gjøres gjennom hele prosessen.

Hvorfor profesjonsetikk henger sammen med kvalitet

- **Mer robuste systemer** – etisk bevissthet gir ofte bedre designvalg.
- **Færre sikkerhets- og personvernbrudd** – risiko håndteres tidligere.
- **Høyere brukertilfredshet** – brukere stoler mer på løsningen.
- **Beskytter omdømme** – både utviklere og virksomheter unngår tillitstap.
- **Reduserer risiko for bias og tillitsbrudd** – færre urettferdige eller skadelige løsninger.

Prosjektplanlegging

Visjonsdokument – hva det typisk inneholder

- **Formål** – hvorfor prosjektet finnes.
- **Omfang** – hva prosjektet skal og ikke skal dekke.
- **Hensikt** – hva man ønsker å oppnå.
- **Målgruppe** – hvem løsningen er for.
- **Funksjonelle krav** – hva systemet skal gjøre.
- **Ikke-funksjonelle krav** – kvalitetskrav som ytelse, sikkerhet og brukervennlighet.
- **Interessenter** – personer eller grupper som påvirkes av prosjektet.
- **Brukerbehov** – behovene løsningen skal dekke.

Risikoanalyse – hva den brukes til

- **Identifisere risiko** – finne mulige problemer.
- **Prioritere risiko** – vurdere hva som er viktigst å håndtere.
- **Definere tiltak** – bestemme hva man skal gjøre med risikoene.
- **Forbedre beslutningsprosesser** – gi bedre grunnlag for valg.

- **Øke prosjektets robusthet** – gjøre prosjektet bedre rustet mot problemer.